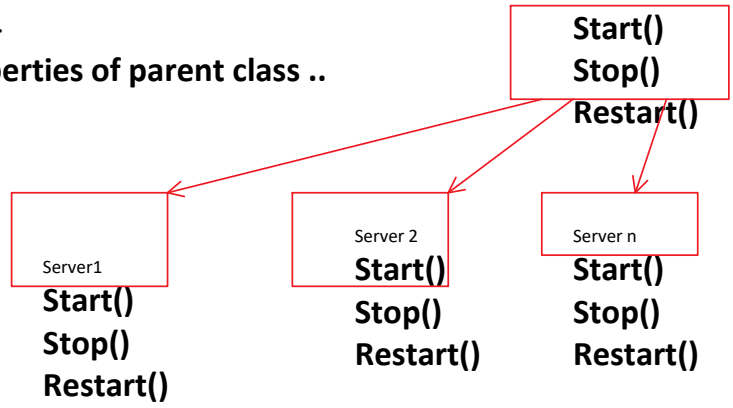
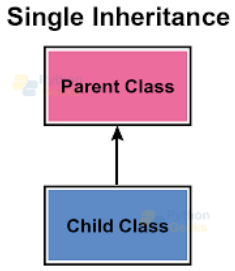
22 April 2026 09:03

Types of inheritances :-
Child class getting properties of parent class ..

Servers..operations
Start()
Stop()
Restart()

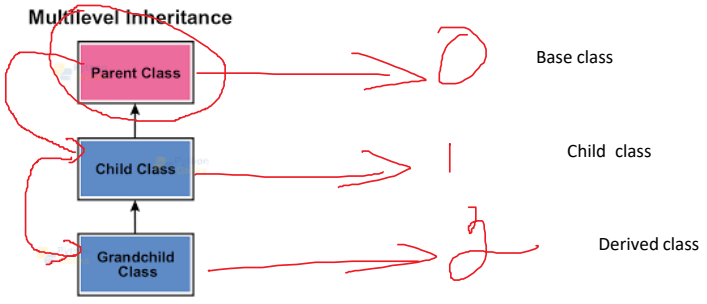


Single inheritance :-



CHILD CLASS GETTING PROPERTIES FROM PARENT CLASS ,..

```
class Server:#parent class
    var="parent class "
    def start(self):
        print("server started")
    def stop(self):
        print("stop server")
class LinuxServer(Server): #child class
    def patch_update(self):
        print("Linux patches installed ")
        print(self.var)
l1=LinuxServer()# creating object for child class only
l1.start()
l1.patch_update()
```



One parent class , multiple child classes ,its in the form of levels

Method resolution order(MRO)--> python searches fro methods starting from the current class and moves upward through the inheritance chain
User

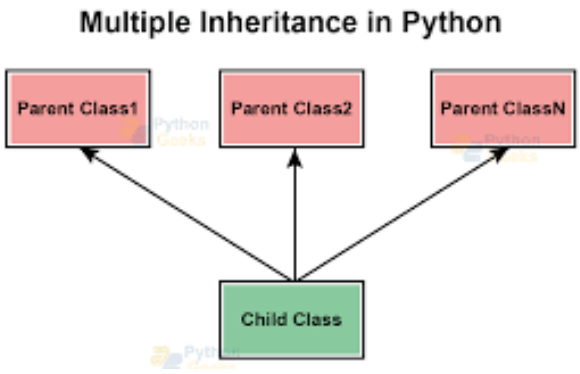
Customer

primeCustomer

primeCustomer

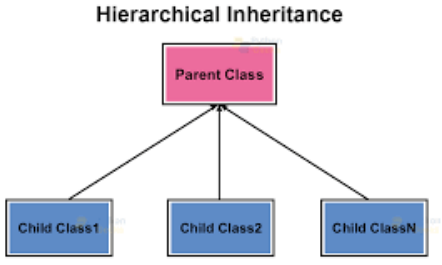
```
# class User: #common login feauture
#     def __init__(self,username):
#         self.username=username
#     def login(self):
#         print(self.username,"logged in successfully")
# class Customer(User): #product purchase
#     def __init__(self, username,customer_id):
#         super().__init__(username)
#         self.customer_id=customer_id
#     def buy_products(self):
#         print(f"Customer Id :{self.customer_id} is purchased product")
# class PrimeCustomer(Customer): # getting extra discount and benefits
#     def __init__(self, username, customer_id,discount):
#         super().__init__(username, customer_id)
#         self.discount=discount
#     def Prime_benefits(self):
#         print(f"prime customer gets : {self.discount} % discount ")
# #object creation:
# obj=PrimeCustomer("Murali",1002,40)
# obj.login()
# obj.buy_products()
# obj.Prime_benefits()
```

Super() in python is used to call methods or constructors of the parent class from child class



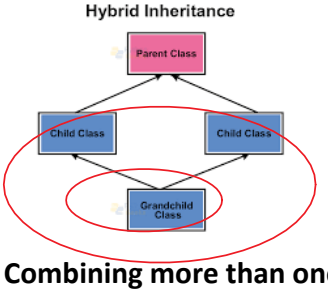
One child class getting properties from different (multiple) parent classes

```
class Logger: #module , parent class 1
    def log(self):
        print("logs captuerd")
class Security:#module  parent class 2
    def Authenticate(self):
        print("User Authenticated ")
class DeploymentTool(Logger,Security): # child class
    def deploy(self):
        print("Application is deployed")
#object creation
t1=DeploymentTool()
t1.log()
t1.Authenticate()
t1.deploy()
```




```
# class CloudProvider: #parent class
#     def login(self):
#         print("cloud login")
#     def billing(self):
#         print("billing enabled ")
# class AWS(CloudProvider):
#       pass
# class GCP(CloudProvider):
```

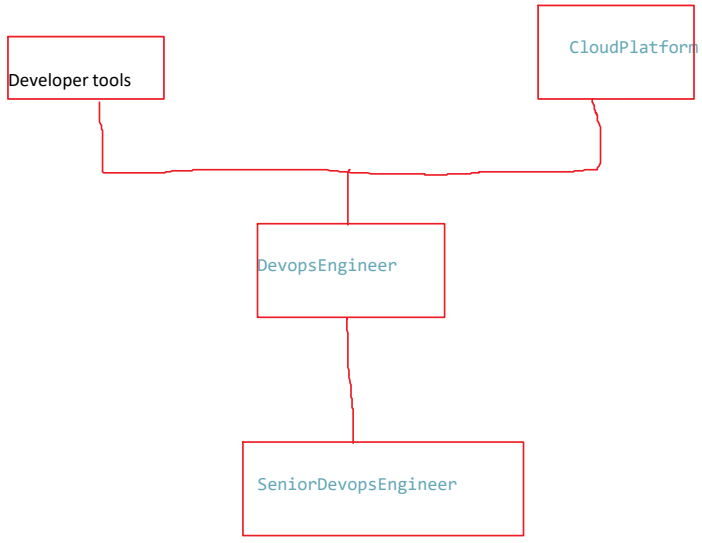
```
#     pass
# class Azure(CloudProvider):
#       print("deployed in Azure")
# obj1=Azure()
# obj1.login()
# obj1.billing()
```



Combining more than one inheritance type

```
class DeveloperTools:#base class1
    def git(self):
        print("using git for version controll")
    def docker(self):
        print("using docker for containerization")
class CloudPlatform:# base classs 2
    def aws(self):
        print("deploying resouurces on aws")
    def kubernaties(self):
        print("managing containers with kubernates ")
class DevopsEngineer(DeveloperTools,CloudPlatform):#dervied class , multiple
inheritence
    def ci_id(self):
        print("Runnin CI/CD pipelines using jenkins ")

class SeniorDevopsEngineer(DevopsEngineer):#multi level  inheritence
    def monitoring(self):
        print("monitoring systems using prometheus & grafana")
emp=SeniorDevopsEngineer()
emp.git()
emp.docker()
emp.aws()
emp.kubernaties()
emp.ci_id()
emp.monitoring()
```




```
# Statick variable :-
# a variable which belongs to the class itself, not to individual object  and we
can also caleed as class variable
class Employee:
    company="wipro" # statick variable
    def __init__(self,name):
        self.name=name

e1=Employee("priya")
print(f'{e1.name} is working with {e1.company}')
Employee.company="clahan" # updating static variables
e2=Employee("akash")
print(f'{e2.name} is working with {e2.company}')
#one varibale can shared by all objects
print(Employee.company) # we can access  static variables without object , by
using class name
```

```
# Static methods:- a method inside a class that doenot use object(self), or
class(cls)
#used for utility or helper functions
# Decorator:
# @staticmethod
# why we need this:-
# when method is logically belongs to the class but doesnt need object data
# example :-
# class Caluculator:
#     @staticmethod
#     def add(a,b):
#         return a+b
# # # c=Caluculator()
# # print(c.add(5,5))
# print(Caluculator.add(10,20))# no need to create an object
#ip address validation
class Utils:
    @staticmethod
    def validate_ip(ip):
        print("ip Validated",ip)
Utils.validate_ip("10.0.0.5")
#date formating
#log formating
# password strength
#port validation
#file extention
```

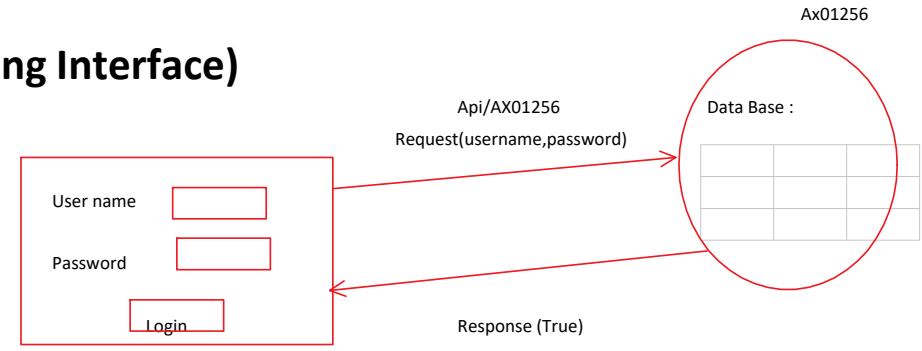
```
# Class methods:-method that works with class variable and class level data..
# Decorator:-
# @classmethod --> instead of self .. cls
# purpose of using class methods are:- used to modify static variables..
#exaample:-
# class Employee:
#     company="clahan"
#     @classmethod
#     def show_comapy(cls,name):
#         cls.company=name

# Employee.show_comapy("wipro") # we arre not creating any object...
# print(Employee.company)
class Server:
    datacenter="DC1"
    @classmethod
    def update_dc(cls,name):
        cls.datacenter=name
Server.update_dc("BangaloreDC1")
print(Server.datacenter)
```

Diff between sels, cls

Keyword	Refer
SELF	CURRENT OBJECT
CLS	CLASS

API(Application Programming Interface)



Having set of rules and protocols that allows different software applications to communicate and interact with each other

Exchanging of data

GET:- retrieve the data from the server

POST:-SEND THE DATA TO THE SERVER(EX, SUBMITING THE FORM

PUT :-UPDATE AN EXISTING RESOURCE

Delete :-Remove the resource

ENDPOINT Http method headers

Common status codes:-\

Code	Meaning
200	Ok (successfully sent request)
201	Created(resource successfully created
400	Bad request (client error)
401	Unauthorized(authentication failed)
404	Not found(requested resource doesnot exists)
500	Internal server error (server error)

PS D:\Myworkspace> pip install requests ---> need to install this package to use API methods